eedi-mining-misconceptions-in-mathematics

5th Place Solution

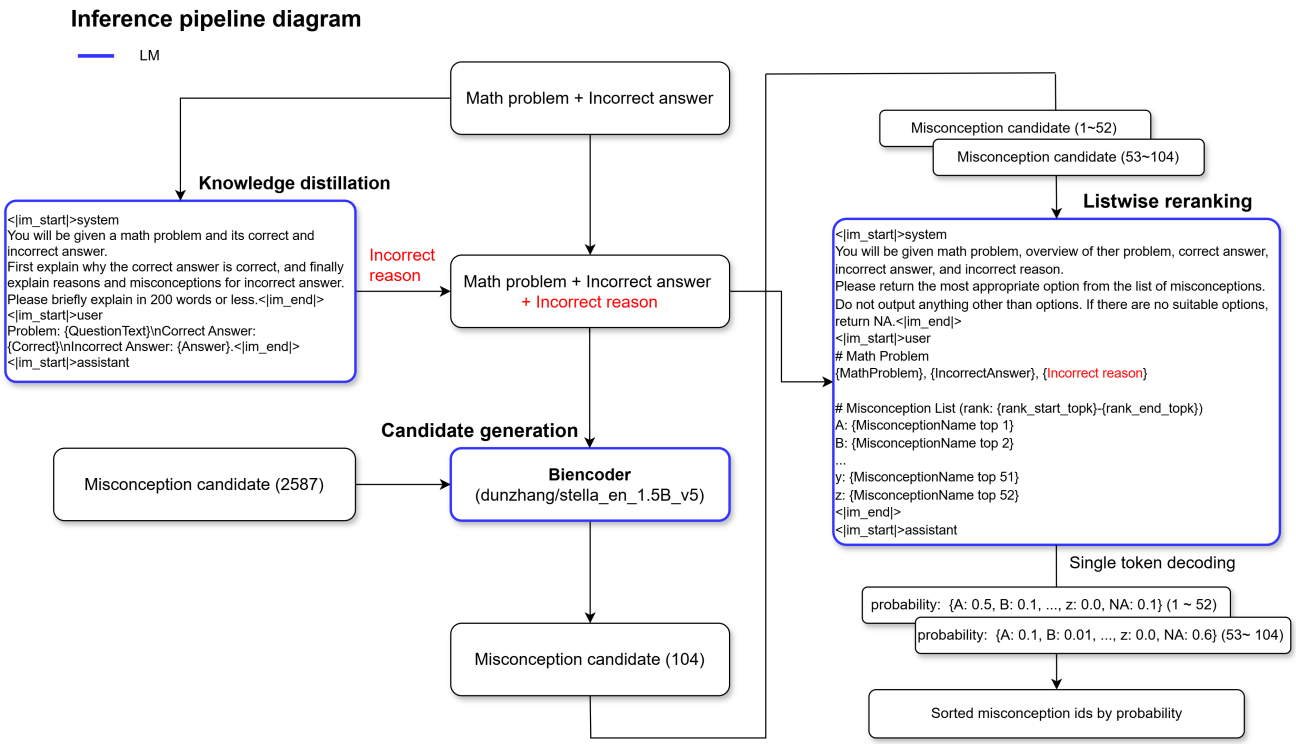
Rank	5
Team	Ebi & ktr
Author	Ebi
Topic ID	551391
Version	Original

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551391
Retrieved with Kaggle CLI on 2026-10-21.

Comment

First, I would like to thank the organizers for hosting such an interesting competition with plenty of room for innovation and no major issues. I also want to thank my teammates who helped create a smooth communication environment for my first team-up experience.

Solution summary



1. **Synthetic data generation**
   - We create problems using LLM with few shot prompting for each MisconceptionId that is not included in the training dataset.
     - For these few shot samples, we insert problems and incorrect answers that have similar MisconceptionNames to the one being generated.
2. **Knowledge Distillation**
   - We input the questiontext, correct answer, and incorrect answer into the LLM (Qwen 2.5 32B Instruct) to generate the reasoning behind the incorrect answer. This content is used as input for all subsequent models (Biencoder, Listwise reranker).
3. **Candidate generation**
   - We use a biencoder model to narrow down the Misconception candidates for each problem and incorrect answer.
4. **Listwise reranking**
   - We input the Misconception candidates sorted by the Biencoder, 52 at a time (corresponding to the number of uppercase and lowercase alphabet letters used as options), into the fine-tuned LLM(Qwen

2.5 32B Instruct) to obtain probability for each option. We then sort the Misconceptions based on these probability.

- For this process, we use the top 104 candidates from the biencoder. In other words, we perform inference twice with 52 candidates each time, extract the probability, and combine the results.

5. **Ensemble**
   - Since the Listwise reranker uses GPTQ + vLLM inference, a single inference takes only about 120 minutes, allowing the results to be ensembled across three folds. (But due to the random delays discussed here, our best sub could not include all results from the third fold. I hope that Kaggle staff will address these issues in the future.)

Detailed solution

1. **Cross validation**
   - Initially, we used GroupKFold(k=5) based on QuestionId, but since the lb was significantly lower than local cv, we suspected there might be many MisconceptionIds that only appear in the test data. Based on this, we switched to GroupKFold(k=5) using SubjectId. This change resulted in a higher proportion of MisconceptionIds that only appear in validation data, which tended to make the cv closer to lb.
     - example local cv 0.626, lb: 0.633
2. **Synthetic data generation**
   - Synthetic data generation is essential to include MisconceptionIds without existing problems in the prediction results. For each MisconceptionName without problems, we generated one set of existing dataset information (QuestionName, SubjectName, ConstructName, Correct Answer, Incorrect Answer) using few shot prompting (4 shots) with LLM (gemini-1.5-pro).
     - For the few shot prompting samples, we embedded all MisconceptionNames using the model (dunzhang/stella_en_1.5B_v5) and included problems corresponding to similar MisconceptionNames. This resulted in approximately +0.01 improvement in cv compared to using random samples.
3. **Knowledge Distillation(KD)**
   - We input the QuestionText, correct answer, and incorrect answer into the LLM (Qwen 2.5 32B Instruct) to estimate the Misconception. Using this information improves the biencoder's cv+0.04. While the Listwise reranker mentioned later uses the same model as KD, it showed an increase of about cv+0.01 compared to when it's not included.
     - prompt

```
PROMPT_FORMAT =
```

4. **Candidate generation**
   - We extract MisconceptionName candidates for each problem and incorrect answer pair using Biencoder. While we perform negative mining, hard negatives were not effective, so we include slightly relaxed samples.

- Training parameters
  - library: SentenceTransfomer
  - model: dunzhang/stella_en_1.5B_v5
  - negative mining params
    - range_min: 512
    - num_negatives: 2
  - lora config
    - r: 32
    - lora_alpha: 64
  - loss: CachedMultipleNegativesRankingLoss
  - epoch: 1
- cv: 0.41, lb: 0.39

5. **Listwise reranker**
   - Overview
     - The reranking part using LLM appears to be the most unique aspect of this competition, and I expect that how this part was implemented significantly impacted the scores. Our teammate @kcotton21 had already come up with the idea of generating single tokens and using their logits for sorting at the beginning of the competition (reference: https://arxiv.org/abs/2406.15657), and our solution is based on this.
   - prompt
     - After various trial and error, we ultimately performed regular fine-tuning using the following prompt to have the LLM generate appropriate choices (which must be single tokens to enable sorting by logits(probability)).

```
NA_PROMPT_FORMAT =
```

     - misconception_list
       - It contains 52 choices of uppercase + lowercase alphabets and their corresponding MisconceptionNames. They are ordered by the Biencoder output.

```
A: Thinks sign  parallel lines just means opposite
B: Does  recognise the notation  parallel sides
.
.
.
y: Thinks that co-interior angles can lie on opposite sides of a transversal
z: Believes the gradient of perpendicular lines just have opposite signs<|im_end|>
```

     - label_choice
       - This is the correct answer choice used for fine-tuning. If the correct answer is not included in the choices, "NA" (which is also a single token in Qwen 2.5 32B) is inserted.
     - rank
       - Indicates which range of choices {start_idx - end_idx} from the Biencoder's output is included.

- Training
  - We divided the top 208 outputs from the biencoder into non-overlapping groups of 52 each (1-52, 53-104, 105-156, 157-208) and performed fine-tuning.
  - Training parameters
    - loss
      - This is the standard cross-entropy loss used in regular fine-tuning, calculated across the entire vocabulary. We chose this simpler implementation method since calculating cross-entropy limited to only the candidate choices + "NA" token showed little difference in results.
    - model: Qwen/Qwen2.5-32B-Instruct
    - lora_config
      - r: 24
      - lora_alpha: 48
    - lr: 5e-5
    - scheduler: cosine
    - warmup_steps: 10
    - epoch : 1.0
    - global batch size: 8
    - library: unsloth
- Inference
  - From the biencoder output, we take the top 104 items and split them into two sets (1-52 and 53-104). Each set is input to the model, generating only single tokens to obtain probabilities for each option. The results from both sets are then combined and sorted.
    - While adding to make it top 156 (52x3 inference) showed a slight improvement in local CV, it wasn't reflected in the leaderboard score, so we didn't adopt this approach.
  - Quantized to 4-bit using GPTQ and inference is performed using vLLM

cv(public)[private] scores

	52x1	**52x2**	**52x3**
fold 4	0.623(0.627)[0572]	0.626(0.633)[0.581]	0.630(0.631)[0.581]
fold 3		(0.644)[0.581]	
fold (3 + 4)		(0.650)[0.590]	
fold (2(60%sample) + 3 + 4)		(0.653)[0.597]	

Computational costs

My local machine is 1 x RTX 4090.

I rented an A100 (from vast.ai) for training a 32B model, and including experiments, it cost me about $350 in total…

Training one 32B Listwise model took about 7 hours on the A100.

What didn't work

- Options used in the Listwise reranker
  - As single token choices, we tried combinations of two alphabets, combinations of alphabets and numbers 0-9, and japanese hiragana, but all of these approaches performed worse than using just the 52 alphabets.
- utilization of QwQ-32B-preview
- multi step reraking (Qwen 2.5 7B(104 candidate) → Qwen 2.5 32B(52 cadindate))
- adding examples of problems and misconceptions in the reraking prompts.

Code release

We released training code. → Eedi-5th-solution

Supplemental Q&A

> **Nikita Babych · 2024-12-13T01:14:23.930000**
>
> Congratulations! I enjoyed competing with you on the public leaderboard.
> It's very interesting that you were able to achieve such a high score without ensembling models on the candidate generation stage. It was quite important in my solution.

> **Ebi · 2024-12-14T08:43:53.017000**
>
> > It's very interesting that you were able to achieve such a high score without ensembling models on the candidate generation stage. It was quite important in my solution.
>
> I think the answer to this is because we reranked the retriever results by 32B model to the top 104, not the top 50 like you did. We were able to obtain a high recall even though the performance of the retriever model was not high.

> **Camaro · 2024-12-14T12:21:21.130000**
>
> Congratulations and thank you for sharing your solution and clean code!

I have a question about data generation.

> For these few shot samples, we insert problems and incorrect answers that have similar MisconceptionNames to the one being generated.

Adding similar MisconceptionNames as few-shot examples is a clever approach. Have you checked how it compares to random sampling?

Ebi · 2024-12-14T12:35:45.937000

I'm glad the code I shared was helpful.

> Adding similar MisconceptionNames as few-shot examples is a clever approach. Have you checked how it compares to random sampling?

Yes, We have confirmed this. There is an effect of about +0.011~0.014 in all of local cv, private lb, and public lb.

Camaro · 2024-12-14T12:37:23.913000

Thanks for the quick answer! I'll give it a try.

A few more questions:

- If I understand your code correctly, it seems to generate one sample per misconception ID. Did generating more samples not improve the score?
- You are using Gemini Flash 1.5. When I tried generating with this model, it sometimes produced incorrect answers(failed to capture misconception). Have you tried other models, such as O1 or al least Gemini Pro?

Ebi · 2024-12-14T12:53:55.260000

> If I understand your code correctly, it seems to generate one sample per misconception ID. Did generating more samples not improve the score?

We tried it, but the local cv got worse, so we didn't submit it. However, after reading some solutions, it seemed important to bring MisconceptionId, which is not included in the trainng data, to the top of the recommendation results, so now I suspect that if we had submitted it, the public lb would have increased.

> You are using Gemini Flash 1.5. When I tried generating with this model, it sometimes produced incorrect answers. Have you tried other models, such as O1 or al least Gemini Pro?

Sorry about this. I made a mistake in sharing the code and we actually used gemini-1.5-pro. We compared the accuracy of biencoder with gemini-1.5-flash and gemini-1.5-pro, but the scores were not different at all (both local cv and public LB).
So we stopped the experiment of changing the LLM used to generate synthetic data here, but now I think it might be effective for Listwise reranking, because there was also no significant change in the biencoder score when we experimented with changing the few-shot examples.

john · 2024-12-19T03:26:22.813000

Congratulations. Thank you for sharing your solution and code! The code is structed very well!
I have some questions about your code for training retriever.

1. hard_negative_params in exp_gpu.yaml
   range_min: 512, num_negatives: 2
   why range_min so large? And only 2 negatives? Can you tell me why you make such decision?
2. train_args
   num_train_epochs: 1.0, per_device_train_batch_size: 640
   Only 1 epoch with so large batch size? Also why you make such decision?

Nikhil Tumbde · 2024-12-19T07:18:17.060000

Hi @ebinan92, thanks for publishing the code and congratulations for securing the gold medal.
I wanted to confirm one thing regarding the quantization, I can see that the re-ranker (32B) was finetuned in BF16, so I believe you directly merged the lora weights with the model weights. Is that right?

Also, any reason for using DataCollatorForSeq2Seq instead of DataCollatorForCompletionOnlyLM ? Am I missing something?

Ebi · 2024-12-20T04:49:32.030000

> I wanted to confirm one thing regarding the quantization, I can see that the re-ranker (32B) was finetuned in BF16, so I believe you directly merged the lora weights with the model weights. Is that right?

Yes, that's correct. We're not using QLoRA to avoid performance degradation during merging.

> Also, any reason for using DataCollatorForSeq2Seq instead of DataCollatorForCompletionOnlyLM ? Am I missing something?

For fine-tuning with unsloth, I'm using DataCollatorForSeq2Seq based on this issue as a reference.
As you mentioned, I think it would also be possible to use DataCollatorForCompletionOnlyLM.

Ebi · 2024-12-20T05:18:03.320000

Short answer:
I chose these parameters because they showed the best recall@100 (my main target metric for bi-encoder) after testing several combinations.

Detailed explanation:

- range_min: 512
  - I adopted a larger value since hard negatives weren't effective with this data.
- num_negatives: 2
  - While increasing this value adds more training data, performance plateaued at around 2, so we set it to 2.
- num_train_epochs: 1.0
  - I set it to 1 because values above 1 led to overfitting.

- per_device_train_batch_size: 640
  - While larger batch sizes are generally better for contrastive learning, increasing it further showed minimal improvement in accuracy, so we kept this value.